

21CSO353T - MOBILE APPLICATION DEVELOPMENT

**SEMESTER VI
(2025-26 EVEN)**

VisionAI – Real Time Object Detection

A PROJECT REPORT

Submitted by

PARNAPALLI ANISH [RA2311004010117]

Under the guidance of
Dr. S KARTHICK



College of Engineering and Technology
SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
S.R.M. Nagar, Kattankulathur – 603203, Chengalpattu District.

APRIL 2026

**SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
KATTANKULATHUR – 603 203**

BONAFIDE CERTIFICATE

Certified that 21CSO353T – Mobile Application Development project report titled “VisionAI – Real Time Object Detection” is the bonafide work of PARNAPALLI ANISH [RA2311004010117], who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr. S KARTHICK

Faculty In-Charge

Dept. of Computational Intelligence

SRM IST, Kattankulathur.

ABSTRACT

VisionAI is a real-time object detection system developed as part of the Mobile Application Development course (21CSO353T) at SRM Institute of Science and Technology. The system leverages lightweight AI models, specifically COCO-SSD and MobileNet implemented via TensorFlow.js, to identify and locate multiple objects in both static images and live video streams. Initially developed as a web application using HTML, CSS, and JavaScript, the project was subsequently converted into a fully functional Android mobile application using Android Studio through WebView integration.

The application offers a comprehensive set of features including image upload with drag-and-drop support, real-time webcam-based detection, bounding box visualization with confidence scores, voice announcements using the Web Speech API, detection history tracking with local storage, and customizable settings for model selection and detection quality. The Android version incorporates necessary device permissions for camera, internet, and storage access to ensure seamless functionality on mobile devices.

This project demonstrates how modern web technologies and lightweight AI models can be effectively combined and deployed on mobile platforms without requiring high-end hardware, making real-time object detection accessible to students, researchers, and developers alike.

Keywords: Object Detection, TensorFlow.js, COCO-SSD, MobileNet, Android WebView, Real-Time Detection, Mobile Application Development.

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO.
	ABSTRACT	iv
	TABLE OF CONTENTS	v
	LIST OF FIGURES	vi
	LIST OF TABLES	vii
1	INTRODUCTION	1
	1.1 Introduction to Project	1
	1.2 Problem Statement	2
	1.3 Motivation	2
	1.4 Sustainable Development Goal of the Project	3
2	LITERATURE SURVEY	4
	2.1 Overview of the Research Area	4
	2.2 Existing Models and Frameworks	4
	2.3 Limitations Identified from Literature Survey	5
	2.4 Research Objectives	6
3	METHODOLOGY	7
	3.1 System Architecture	7
	3.2 Block Diagram	7
	3.3 Component Description	8
	3.4 Tools and Software Used	9
4	RESULTS AND DISCUSSION	10
	4.1 Upload Image Interface	10
	4.2 Webcam Detection Interface	10
	4.3 Detection Results Interface	11
	4.4 History Tracking Interface	11
	4.5 Settings Interface	12
5	CONCLUSION	13
	REFERENCES	14

LIST OF FIGURES

Figure No.	Title	Page No.
Figure 3.1	Block Diagram of VisionAI Android Application Architecture	7
Figure 4.1	Upload Image Interface with Bounding Box Detection	10
Figure 4.2	Webcam Detection Interface (Android App)	10
Figure 4.3	Detection Results Interface showing Metrics and Objects	11
Figure 4.4	History Tracking Interface with Detection Logs	11
Figure 4.5	Settings Interface with Configuration Options	12

LIST OF TABLES

Table No.	Title	Page No.
Table 3.1	Tools and Software Used in VisionAI Development	9

CHAPTER 1

INTRODUCTION

1.1 Introduction to Project

In recent years, Artificial Intelligence (AI) and Computer Vision have become essential technologies for enabling machines to understand and interpret visual data. One of the most significant applications of computer vision is object detection, which allows systems to identify and locate multiple objects in images and real-time video streams. This technology is widely deployed in areas such as security surveillance, smart devices, healthcare, and factory automation.

The VisionAI project is developed to create a simple, efficient, and user-friendly object detection system that can be used by students and developers without requiring complex setups or high-end hardware. The system uses lightweight AI models, specifically COCO-SSD and MobileNet, which are capable of performing fast and accurate detection directly within the application without sending data to external servers.

VisionAI was initially designed as a web-based application using HTML, CSS, and JavaScript, where object detection was implemented using TensorFlow.js. The application supports features such as image upload, real-time webcam detection, bounding box visualization, result analysis, voice announcements, customizable settings, and history tracking. The web application was further converted into an Android application using Android Studio through WebView integration, allowing it to run seamlessly within a mobile environment with all required device permissions configured.

1.2 Problem Statement

In today's digital era, object detection plays a vital role in many real-world applications such as security surveillance, smart cities, healthcare monitoring, retail automation, and autonomous systems. Despite its importance, the practical adoption of object detection remains limited due to several key challenges:

- Most existing systems require high computational power and specialized hardware, making them expensive and inaccessible to a wide range of users.
- Available tools are often complex and not beginner-friendly, requiring advanced knowledge in machine learning and programming.
- There is a lack of simple, interactive platforms where users can easily test object detection in real-time or on custom images without complex installation.
- Many systems do not provide features like history tracking or easy visualization, which are important for learning, analysis, and demonstration.

To overcome these limitations, there is a need for a lightweight, user-friendly, and accessible object detection system that can run efficiently on everyday devices while providing real-time interaction and meaningful analytical insights.

1.3 Motivation

The primary motivation behind VisionAI is to make AI-powered object detection accessible to everyone, regardless of their hardware capabilities or technical background. Traditional deep learning systems demand powerful GPUs and complex environment setups, creating a high barrier to entry for students and beginner developers. By leveraging browser-based AI with TensorFlow.js and deploying it as an Android application through WebView, VisionAI removes these barriers and enables real-time object detection on ordinary smartphones and laptops.

The project is also motivated by the growing need to bridge the gap between academic learning and practical implementation. Students studying mobile application development need hands-on exposure to integrating AI capabilities into apps. VisionAI provides a complete, end-to-end example of how a web-based AI system can be packaged into a native Android app, covering aspects of permissions management, WebView configuration, and on-device model inference.

Furthermore, the inclusion of voice feedback, history tracking, and configurable settings makes VisionAI suitable not just as a learning tool, but also for real-world exploratory use cases in fields like assistive technology, smart retail, and educational demonstrations.

1.4 Sustainable Development Goal of the Project

VisionAI aligns with the United Nations Sustainable Development Goals (SDGs), particularly:

- **SDG 4 – Quality Education:** VisionAI serves as a practical, hands-on learning platform for students and developers exploring AI and mobile application development. By removing hardware barriers and providing an interactive interface for real-time experimentation, it promotes inclusive and quality technical education.
- **SDG 9 – Industry, Innovation and Infrastructure:** The project demonstrates a cost-effective and innovative approach to deploying AI on mobile devices using lightweight models and web-to-app conversion, contributing to the goal of building resilient and inclusive technological infrastructure.
- **SDG 10 – Reduced Inequalities:** By running on ordinary devices without specialized hardware, VisionAI ensures that AI-powered object detection is accessible to users across different socioeconomic backgrounds, helping to reduce the technology access gap.

CHAPTER 2

LITERATURE SURVEY

2.1 Overview of the Research Area

Object detection is a fundamental problem in computer vision that involves identifying and localizing multiple objects within digital images or video frames. Unlike image classification, which assigns a single label to an entire image, object detection produces bounding boxes along with class labels and confidence scores for each detected instance. The field has advanced significantly over the past decade, transitioning from handcrafted feature-based methods (such as Histogram of Oriented Gradients combined with Support Vector Machines) to deep learning approaches that learn hierarchical feature representations directly from data.

Modern object detection models can be broadly classified into two categories: two-stage detectors (such as Faster R-CNN), which first propose candidate regions and then classify them, and one-stage detectors (such as YOLO and SSD), which predict bounding boxes and class probabilities in a single pass. One-stage detectors are preferred in real-time and mobile applications due to their significantly lower inference latency. The introduction of lightweight architectures like MobileNet has further enabled deployment on resource-constrained devices such as smartphones and embedded systems.

2.2 Existing Models and Frameworks

Several key models and frameworks have been reviewed to understand the landscape of object detection and inform the design of VisionAI:

YOLO (You Only Look Once): Proposed by Redmon et al. [1], YOLO redefined real-time object detection by reformulating the detection task as a single regression problem, predicting bounding box coordinates and class probabilities simultaneously from the full image in one evaluation. This approach enabled extremely fast inference, making it suitable for video analysis and surveillance systems. However, the original YOLO architecture was not designed for mobile deployment due to its model size.

MobileNet: Developed by Howard et al. at Google [2], MobileNet introduced depthwise separable convolutions as a means of drastically reducing the number of computations and model parameters compared to standard convolutional networks. This made MobileNet one of the first CNN architectures specifically designed for mobile and embedded vision applications. MobileNet is used in VisionAI as a backbone for image classification and lightweight detection, offering a strong accuracy-efficiency trade-off.

SSD – COCO-SSD: Liu et al. [3] proposed the Single Shot MultiBox Detector (SSD), which eliminates the need for a region proposal stage by directly predicting object categories and offsets for a fixed set of default bounding boxes at multiple scales within a single network pass. The COCO-SSD variant is pre-trained on the COCO dataset (80 object categories) and is the primary detection model used in VisionAI due to its availability in TensorFlow.js and its balance between speed and detection accuracy.

TensorFlow.js: Smilkov et al. [4] presented TensorFlow.js, a JavaScript library that enables machine learning model training and inference directly in the web browser using WebGL acceleration or in Node.js. TensorFlow.js supports loading pre-trained models from TensorFlow Hub and the TensorFlow.js Model Hub, enabling client-side AI inference without server-side computation. VisionAI uses TensorFlow.js to run COCO-SSD and MobileNet directly in the browser, enabling real-time detection without any backend dependency.

2.3 Limitations Identified from Literature Survey (Research Gaps)

The review of existing work highlights the following limitations and gaps that VisionAI is designed to address:

- Most high-accuracy object detection models such as Faster R-CNN and full-size YOLO variants require dedicated GPU hardware for real-time inference, making them impractical for everyday consumer devices.
- While TensorFlow Lite enables on-device inference for Android, existing tutorials and implementations are often fragmented, with limited integration of a complete user-facing application including history tracking, voice feedback, and customizable settings.
- Existing browser-based detection demos lack the ability to be seamlessly packaged into a native Android application that preserves all web functionality through WebView, a gap that VisionAI specifically addresses.
- No unified platform was found that combines multi-model detection (COCO-SSD + MobileNet together), voice announcements, confidence threshold tuning, and local history tracking in a single lightweight mobile-compatible application.

2.4 Research Objectives

Based on the literature review and identified research gaps, the following research objectives were formulated for the VisionAI project:

- To develop a web-based real-time object detection application using TensorFlow.js with COCO-SSD and MobileNet models that runs entirely in the browser without server-side processing.
- To implement a complete user-facing interface with image upload, live webcam detection, bounding box visualization, voice feedback, and configurable detection settings.
- To design and implement a local history tracking system that logs detection results, thumbnails, and metadata for analysis and review.
- To convert the web application into a fully functional Android mobile application using Android Studio with WebView integration, ensuring all features operate correctly on mobile devices.
- To evaluate the system's detection accuracy, processing latency, and usability across different input modes and device configurations.

CHAPTER 3

METHODOLOGY

3.1 System Architecture

The VisionAI system follows a layered architecture that integrates web-based AI capabilities within a native Android shell. The system accepts input through either the device camera for real-time detection or the media gallery for static image analysis. Input is processed by lightweight on-device AI models, detection results are rendered as visual overlays with bounding boxes and labels, and all results are stored locally for history tracking. User-configurable settings govern model selection, confidence thresholds, voice output, and display preferences throughout the entire pipeline.

The web application layer communicates with the Android host through the WebView bridge, which grants the web content access to native device capabilities such as the camera, storage, and internet. This architecture ensures that the full feature set of the web application remains intact within the Android wrapper while benefiting from native permission management and device API access.

3.2 Block Diagram

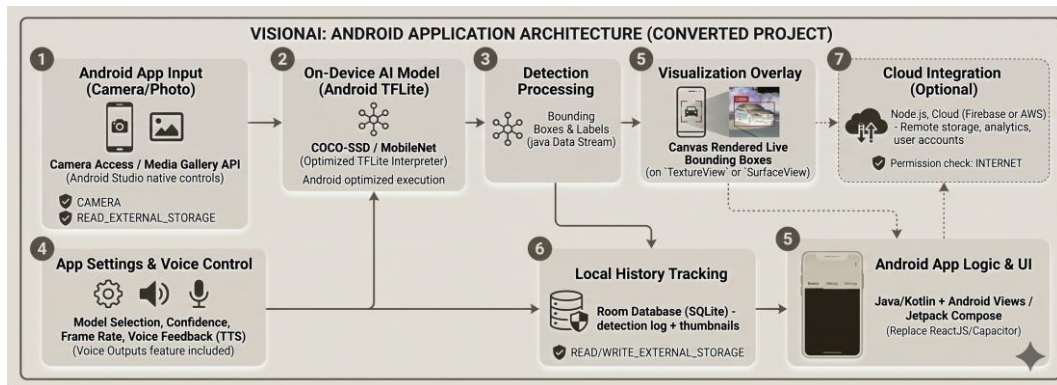


Figure 3.1: Block Diagram of VisionAI Android Application Architecture

The block diagram illustrates the complete data flow of VisionAI. The system begins with Android App Input (Camera / Photo), which feeds into the On-Device AI Model running COCO-SSD and MobileNet via TensorFlow Lite. The Detection Processing stage generates bounding boxes and labels, which are rendered on screen through the Visualization Overlay. App Settings and Voice Control govern user preferences, while all detection data is persisted through Local History Tracking using Room (SQLite). The Android App Logic and UI layer integrates all components, with optional Cloud Integration available for extended functionality.

3.3 Component Description

1. Android App Input (Camera / Photo): The entry point of the application captures input through the device camera (real-time) or from the media gallery (image upload) using native Android APIs. Permissions CAMERA and READ_EXTERNAL_STORAGE are required.

2. On-Device AI Model (TensorFlow Lite): Input frames are processed by COCO-SSD and MobileNet models running on-device via TensorFlow Lite. On-device inference ensures low latency, faster processing, and enhanced privacy since no data is transmitted to external servers.

3. Detection Processing: The AI model processes input images or live frames to generate bounding box coordinates, class labels, and confidence scores. This runs continuously for live video streams or once per uploaded image, forming the core intelligence of the system.

4. Visualization Overlay (Bounding Boxes + Labels): Detection results are rendered on-screen using Canvas, TextureView, or SurfaceView to create an augmented live view. Each detected object is highlighted with a bounding box, label, and confidence score, providing immediate visual feedback.

5. App Settings & Voice Control: Users can configure model selection, confidence threshold, detection quality mode (Fast, Balanced, High Quality), and enable Text-to-Speech voice announcements. These settings directly influence model behavior and result presentation.

6. Local History Tracking: All detection results including thumbnails, object counts, average confidence, processing time, and labels are persisted locally using Room (SQLite). This enables users to review and compare past detections without network access.

7. Android App Logic & UI: Built using Java/Kotlin with Android Views or Jetpack Compose, this layer manages navigation, user interactions, and integration of all components. It replaces the original React/Capacitor frontend while preserving all original functionality.

8. Cloud Integration (Optional): An optional Firebase or AWS integration component enables remote storage, analytics, and user account management. The core VisionAI application functions fully offline; cloud integration enhances scalability for advanced deployments.

3.4 Tools and Software Used

The VisionAI project was developed using a combination of web technologies and Android development tools. The web interface was built with HTML, CSS, and JavaScript, with TensorFlow.js powering AI model inference. Web APIs handled webcam access, bounding box rendering, voice feedback, and local history storage. The web application was then packaged into a native Android application using Android Studio with WebView integration, enabling deployment on Android devices with all required permissions.

Category	Tool / Software Used	Purpose
Development IDE	Android Studio	Convert website to Android app and build APK
Programming Languages	HTML, CSS, JavaScript	Develop the VisionAI web application
AI Framework	TensorFlow.js	Run AI models directly in the browser
AI Models	COCO-SSD, MobileNet	Object detection and image classification
Web APIs	WebRTC (getUserMedia)	Access webcam for real-time detection
Browser Features	Canvas API	Draw bounding boxes and detection overlays
Voice Feature	Web Speech API	Provide voice output for detected objects
Storage	LocalStorage / Room (SQLite)	Store detection history locally
Web-to-App	Android WebView	Run website inside Android application
Permissions	Camera, Internet, Storage	Enable webcam, model loading, file access
Testing	Android Emulator / Real Device	Test application functionality

Table 3.1: Tools and Software Used in VisionAI Development

CHAPTER 4

RESULTS AND DISCUSSION

The VisionAI application was successfully developed, tested, and validated across both the web platform and Android devices. All core features including image-based detection, real-time webcam detection, history tracking, and settings customization were confirmed to function as expected. The following sections present the output screenshots from each interface module of the application.

4.1 Upload Image Interface

The upload image module allows users to provide static images for object detection either by dragging and dropping an image onto the designated area or by manually browsing and selecting a file from the device. Once an image is submitted, the system displays it on-screen and enables three action buttons: Detect Objects (standard single-model detection), Enhanced Detection (multi-model mode using both COCO-SSD and MobileNet simultaneously), and Reset. Detected objects are highlighted with labeled bounding boxes and confidence percentage scores, providing a clear visual representation of detection results. This interface acts as the primary entry point for static image-based object detection.

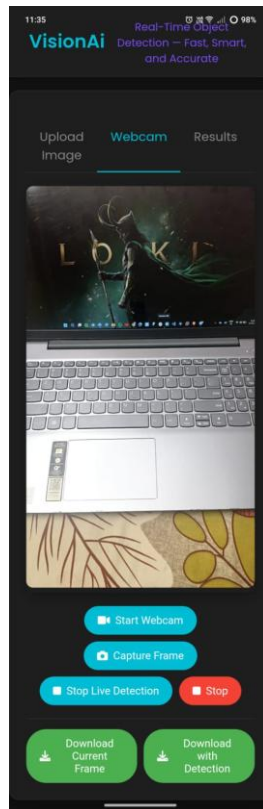


Figure 4.1: Upload Image Interface with Bounding Box Detection

4.2 Webcam Detection Interface

The webcam detection module integrated into the Android application enables real-time video-based object detection using the device camera. Users can interact with the camera through control buttons: Start Webcam to initialize the camera feed, Capture Frame to take a snapshot, Live Detection to begin continuous frame-by-frame analysis, and Stop to end the

session. Additional download options allow users to save the current frame or the annotated detection output to local storage. The interface demonstrates the successful deployment of real-time object detection capabilities within the Android WebView environment.

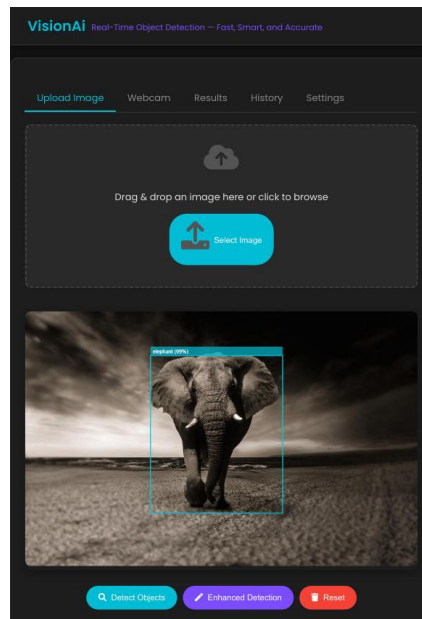


Figure 4.2: Webcam Detection Interface (Android App)

4.3 Detection Results Interface

The results page presents a comprehensive analytical view of the most recent detection run. Key performance metrics displayed at the top include the total number of objects detected, the average confidence score as a percentage, and the total processing time in milliseconds. Below the metrics, each detected object is listed individually with its class label, confidence level displayed as a progress bar, and the model tag (COCO-SSD or MobileNet) responsible for the detection. A prominently placed "Save to History" button allows users to manually persist the current results to the local history database for future reference and comparison.

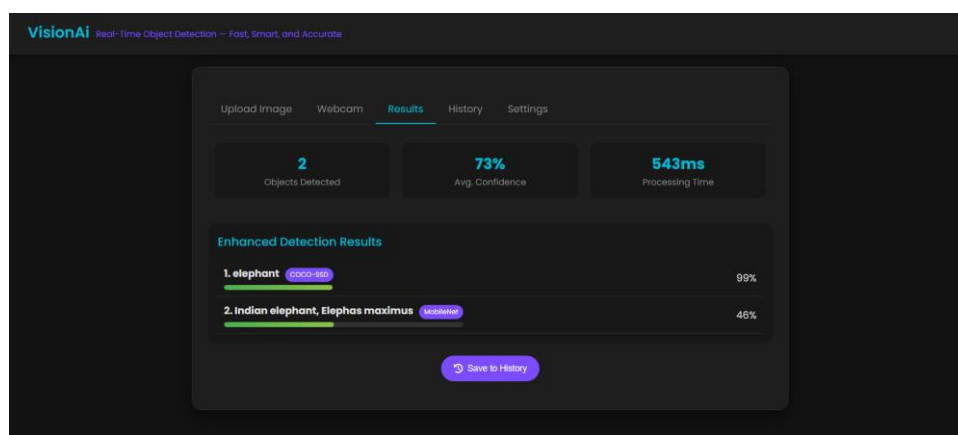


Figure 4.3: Detection Results Interface showing Performance Metrics and Detected Objects

4.4 History Tracking Interface

The history module provides a persistent log of all previously saved detection sessions. Each history entry displays the captured image thumbnail, detection timestamp, number of objects detected, average confidence score, processing duration, and all detected object labels.

A summary statistics panel at the top of the history page aggregates data across all saved sessions, showing the total number of saved detections, cumulative objects detected, and the most frequently detected object class. This feature is implemented using local storage (browser) and Room SQLite (Android), enabling review and comparison of past results without any internet connectivity.



Figure 4.4: History Tracking Interface with Detection Logs and Summary Statistics

4.5 Settings Interface

The settings module provides full control over detection behavior and display preferences through an intuitive toggle and slider-based interface. Detection Confidence: a slider to set the minimum confidence threshold below which detections are suppressed. Detection Models: a dropdown to select Auto (recommended), COCO-SSD, MobileNet, or all models. Detection Quality: a selector for Fast, Balanced, or High Quality processing modes. Display toggles include Show Bounding Boxes to enable or disable rectangular overlays, and Show Object Details to show or hide label text on detections. Advanced features include Voice Announcements (TTS audio feedback), Multi-Model Detection (combined inference), Auto-Save to History (automatic logging without manual action), and History Limit (maximum number of stored records to manage device storage efficiently).

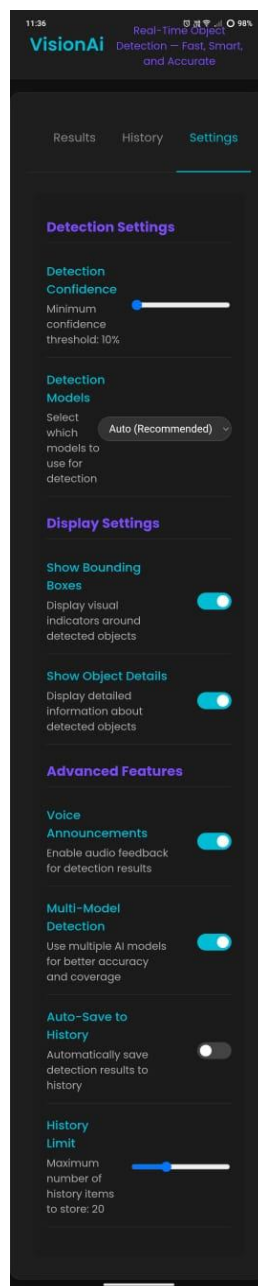


Figure 4.5: Settings Interface with Detection and Display Configuration Options

CHAPTER 5

CONCLUSION

The VisionAI project successfully demonstrates the end-to-end development of a real-time object detection system and its transformation from a web-based application into a fully functional Android mobile application using Android Studio. The system integrates advanced AI models, COCO-SSD and MobileNet, powered by TensorFlow.js, to perform efficient and accurate object detection on both static images and live webcam input. By leveraging browser-based technologies and packaging them within an Android WebView container, the project highlights how modern web applications can be effectively adapted for mobile deployment with minimal architectural changes.

The application delivers a complete and polished user experience through features such as image upload, real-time webcam detection, bounding box visualization, voice feedback via Web Speech API, customizable detection settings, and persistent history tracking using local storage and Room SQLite. These features collectively enhance usability and make the system suitable for learning, demonstration, and practical applications across multiple domains. The user-controlled settings panel allows flexibility in balancing detection speed against accuracy, while the history module adds significant value for analytical review and comparison over time.

From a mobile application development perspective, the project underscores the importance of Android permissions management, the integration of web technologies into native applications through WebView, and optimization strategies for achieving real-time performance on resource-constrained devices. The use of lightweight AI models eliminates the dependency on high-end hardware, making AI-powered object detection truly accessible on everyday smartphones.

Overall, VisionAI stands as a strong integrative example of Artificial Intelligence, Web Development, and Mobile Application Development converging into a single unified system. It is valuable as an academic project, a learning platform, and a foundation for practical computer vision applications, successfully bridging the gap between complex AI research and everyday usability.

REFERENCES

- [1] Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You Only Look Once: Unified, Real-Time Object Detection. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, USA, pp. 779–788. DOI: 10.1109/CVPR.2016.91
- [2] Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., & Adam, H. (2017). MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. arXiv preprint arXiv:1704.04861. [Online]. Available: <https://arxiv.org/abs/1704.04861>
- [3] Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C. Y., & Berg, A. C. (2016). SSD: Single Shot MultiBox Detector. European Conference on Computer Vision (ECCV), Amsterdam, Netherlands, Lecture Notes in Computer Science, vol. 9905, pp. 21–37. DOI: 10.1007/978-3-319-46448-0_2
- [4] Smilkov, D., Carter, S., Sculley, D., Viégas, F. B., & Wattenberg, M. (2019). TensorFlow.js: Machine Learning for the Web and Beyond. Proceedings of the 2nd SysML Conference, Palo Alto, CA, USA. arXiv preprint arXiv:1901.05350. [Online]. Available: <https://arxiv.org/abs/1901.05350>